SCIENCES SORBONNE UNIVERSITÉ

FORMALISATION AND EVALUATION OF PROPERTIES FOR CONSEQUENTIALIST MACHINE ETHICS

Thursday 16th May, 2024

Titouan Guerin

LIP6

MLIA Machine Learning & Deep Learning for Information Access

Introduction

Contexte

SCIENCES SORBONNE UNIVERSITÉ

Motivation

- AI systems increasingly make decisions that impact humans' lives (self-driving cars)
- Need to follow ethical principles

Contexte

SCIENCES SORBONNE UNIVERSITÉ

Motivation

- AI systems increasingly make decisions that impact humans' lives (self-driving cars)
- Need to follow ethical principles

Problem

- Existing approaches lack rigorous justification and formal comparison.

Ethics

SCIENCES SORBONNE UNIVERSITÉ

There are 3 main approaches to ethics:

Ethics

There are 3 main approaches to ethics:

- consequentialism: judges actions only based on their **consequences**
- deontology: notion of **duty** to determine whether an action/ sequence of actions is right or wrong
- virtue ethics: uses agent's **character and intention** to determine the appropriateness of their actions

Ethics

SCIENCES SORBONNE UNIVERSITÉ

There are 3 main approaches to ethics:

- consequentialism: judges actions only based on their **consequences**
- deontology: notion of **duty** to determine whether an action/ sequence of actions is right or wrong
- virtue ethics: uses agent's **character and intention** to determine the appropriateness of their actions

We will focus on **consequentialism** because it is the dominant approach to machine ethics so far and the focus of this paper.

Problematique

SCIENCES SORBONNE UNIVERSITÉ

Question

How can machine ethics approaches properly validate their formalisation of ethical principles and determine the most appropriate principle(s) to apply in a given situation?

Problematique

SCIENCES SORBONNE UNIVERSITÉ

Question

How can machine ethics approaches properly validate their formalisation of ethical principles and determine the most appropriate principle(s) to apply in a given situation?

Contributions

- Framework to evaluate ethical principles systematically,
- Define formal principles and properties for decision-making.

Running example

SCIENCES SORBONNE UNIVERSITÉ

We have available to us exactly ***one smart robot****, designed to assist in decision-making when "incidents" occur. A* ***fire just started*** *on the top floor of a building* ***full of employees****. The robot needs to* ***keep everyone safe*** *but also to* ***minimise the damage by extinguishing the fire****. It has three choices:*

- *(a) rush into the source of the fire and extinguish it,*
- *(b) personally help the evacuation of employees until everyone is safe,*
- *(c) trigger the fire alarm while calling the fire department.*

What should it do?

Situation Calculus

Situation Calculus

What is it?

- Formal framework to model and reason about changing environments

Situation Calculus

SCIENCES SORBONNE UNIVERSITÉ

What is it?

- Formal framework to model and reason about changing environments
- Compute what happens if an agent performs a sequence of actions

Situation Calculus

What is it?

- Formal framework to model and reason about changing environments
- Compute what happens if an agent performs a sequence of actions

Lets define the framework!

Definitions

Fluents

What they are: Properties of the world that can change over time,
Example: "The building is on fire", "Employees are safe", "The fire is extinguished",
Formal notation: $F(\vec{x}, s)$ means "property F with arguments $\vec{x}$ is true in situation s."

Definitions

Fluents

What they are: Properties of the world that can change over time,
Example: "The building is on fire", "Employees are safe", "The fire is extinguished",
Formal notation: $F(\vec{x}, s)$ means "property F with arguments $\vec{x}$ is true in situation s."

Situations

What they are: Snapshots of the world at a given time, including the history of actions that led to it,
Example:

- s_0: Initial situation (building on fire, employees inside)
- $do(a, s)$: The situation after performing action a in situation s.

Formal notation: $do([evac, extg], s_0)$ means the situation after evacuating and then extinguishing the fire.

Definitions

Actions

What they are: Things that change the world,
Example: *evac* (evacuate employees), *extg* (extinguish fire), *call* (call the fire department).

Definitions

SCIENCES SORBONNE UNIVERSITÉ

Actions

What they are: Things that change the world,
Example: *evac* (evacuate employees), *extg* (extinguish fire), *call* (call the fire department).

Action Preconditions

What they are: Conditions that must be true for an action to be possible.
Example: To *extg*, the robot must be near the fire source.
Formal notation: $Poss(a(\vec{x}), s) \equiv \pi_a(\vec{x}, s)$ means "action a is possible in situation s if π_a is true."

Definitions

Effect Axioms

What they are: Rules that describe how actions change fluents.

- Positive effect: F becomes true after a.
- Negative effect: F becomes false after a.

Example:

- $do(evac, s)$ makes "employees are safe" true.
- $do(extg, s)$ makes "building is on fire" false.

Formal notation:

- $\gamma_F^+(\vec{x}, a, s) \rightarrow F(\vec{x}, do(a, s))$ (positive effect)
- $\gamma_F^-(\vec{x}, a, s) \rightarrow \neg F(\vec{x}, do(a, s))$ (negative effect)

Definitions

Successor State Axiom

What it is: A rule that fully describes how a fluent changes after an action,

Example: If *extg* is performed, "building is on fire" becomes false, and all other fluents stay the same unless changed,

Formal notation:

$$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee (F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s))$$

Definitions

Successor State Axiom

What it is: A rule that fully describes how a fluent changes after an action,
Example: If *extg* is performed, "building is on fire" becomes false, and all other fluents stay the same unless changed,
Formal notation:
$F(\vec{x}, do(a, s)) \equiv \gamma_F^+(\vec{x}, a, s) \vee (F(\vec{x}, s) \wedge \neg\gamma_F^-(\vec{x}, a, s))$

Plans

What they are: Sequences of actions,
Example: $\pi = [evac, extg, call]$,
Formal notation: $do(\pi, s_0)$ is the situation after executing plan π from s_0.

Updated Example

Consider an initial state of our example where the building is on fire and all employees are inside. The robot has options to extinguish the fire (extg), evacuate all employees (evac), and call the fire department (call). The employees will get injured if not evacuated immediately and the building will become more damaged the longer it is not extinguished.

Updated Example

Consider an initial state of our example where the building is on fire and all employees are inside. The robot has options to extinguish the fire (extg), evacuate all employees (evac), and call the fire department (call). The employees will get injured if not evacuated immediately and the building will become more damaged the longer it is not extinguished.

Let's try and give some examples using the previous definition!

Examples

In s_0, we can say that:

- $OnFire(s0)$: True (the building is on fire),
- $EmployeesSafe(s0)$: False (employees are not safe),
- $FireExtinguished(s0)$: False (the fire is not extinguished),

Examples

In s_0, we can say that:

- $OnFire(s0)$: True (the building is on fire),
- $EmployeesSafe(s0)$: False (employees are not safe),
- $FireExtinguished(s0)$: False (the fire is not extinguished),

And some action preconditions could be:

- $Poss(extg, s) \equiv OnFire(s)$: The robot can extinguish the fire only if the building is on fire,
- $Poss(evac, s) \equiv \neg EmployeesSafe(s)$: The robot can evacuate employees only if they are not already safe.

Examples

In s_0, we can say that:

- $OnFire(s0)$: True (the building is on fire),
- $EmployeesSafe(s0)$: False (employees are not safe),
- $FireExtinguished(s0)$: False (the fire is not extinguished),

And some action preconditions could be:

- $Poss(extg, s) \equiv OnFire(s)$: The robot can extinguish the fire only if the building is on fire,
- $Poss(evac, s) \equiv \neg EmployeesSafe(s)$: The robot can evacuate employees only if they are not already safe.

And one positive effect axiom could be:

- $\gamma^{+}_{FireExtinguished}(extg, s) \rightarrow FireExtinguished(do(extg, s))$ which means that performing the extinguish action in s always results in *FireExtinguished* to become True.

Formalisation of Ethical Principles

Definition

SCIENCES SORBONNE UNIVERSITÉ

Overview

Objective here is not to achieve a **goal**, but rather to see if a plan satisfies an **ethical principle**.

Definition

Overview

Objective here is not to achieve a **goal**, but rather to see if a plan satisfies an **ethical principle**.

Goodness

We define a plan as **ethically permissable** based on its *goodness*.
We introduce the function $goodness(state(s))$ which returns a *value*.
We also introduce $harmful(\varphi)$ which is true if φ is a harmful fluent.

Definition

Overview

Objective here is not to achieve a **goal**, but rather to see if a plan satisfies an **ethical principle**.

Goodness

We define a plan as **ethically permissable** based on its *goodness*.
We introduce the function $goodness(state(s))$ which returns a *value*.
We also introduce $harmful(\varphi)$ which is true if φ is a harmful fluent.

We simplify the notation for *admissible* plans and goodness:

- $state(\pi) = state(do(\pi, s_0))$
- $goodness(\pi) = goodness(state(\pi))$

Admissibility

Ethical Admissibility

We define a plan π as **ethically admissible** if it is both possible and satisfies an ethical principle.

The paper defines 5 ethical principles: **utilitarianism**, **hedonism**, **no new harm**, **remove harm** and **harm avoidance**.
Let's define them formally!

Ethical Principles

Utilitarianism

It focuses on bringing the greatest good to the greatest number of people:

$$\mathcal{EP}_{util}(\Pi, s_0) = \{\pi \in \Pi \mid goodness(\pi) \geq goodness(\pi') \text{ for all } \pi' \in \Pi\}$$

Ethical Principles

Utilitarianism

It focuses on bringing the greatest good to the greatest number of people:

$$\mathcal{EP}_{util}(\Pi, s_0) = \{\pi \in \Pi \mid goodness(\pi) \geq goodness(\pi') \text{ for all } \pi' \in \Pi\}$$

Example:
If $goodness([evac, extg]) > goodness([extg, call])$, then $[evac, extg]$ is ethically permissible.

Ethical Principles

Hedonism

Based on the overall consequences that it produces, a direct interpretation of consequentialism: any plan is ethically permissible if it has more goodness at the end state compared to the initial state.

$$\mathcal{EP}_{hedon}(\Pi, s_0) = \{\pi \in \Pi \mid goodness(\pi) \geq goodness(\emptyset)\}$$

Ethical Principles

Hedonism

Based on the overall consequences that it produces, a direct interpretation of consequentialism: any plan is ethically permissible if it has more goodness at the end state compared to the initial state.

$$\mathcal{EP}_{hedon}(\Pi, s_0) = \{\pi \in \Pi \mid goodness(\pi) \geq goodness(\emptyset)\}$$

Example:
If $goodness([evac]) \geq goodness(\emptyset)$, then $[evac]$ is ethically permissible.

Ethical Principles

No New Harm

It dictates that a plan should not cause any new harm, so an agent should not be directly(or actively) responsible for any harm. This also means that we tolerate a harm that is unavoidable.

$$\mathcal{EP}_{\text{nnHarm}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi) \text{ and } \mathit{state}(s_0) \nvdash \varphi, \text{ then } \big(\mathit{state}(\pi) \nvdash \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\big)\}$$

Ethical Principles

No New Harm

It dictates that a plan should not cause any new harm, so an agent should not be directly(or actively) responsible for any harm. This also means that we tolerate a harm that is unavoidable.

$$\mathcal{EP}_{\text{nnHarm}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi) \text{ and } \mathit{state}(s_0) \nvdash \varphi, \text{ then } \big(\mathit{state}(\pi) \nvdash \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\big)\}$$

Explanation:
For every harmful predicate φ,

If φ is not true in the initial state ($\mathit{state}(s_0) \nvdash \varphi$), Then φ must not become true after executing π ($\mathit{state}(\pi) \nvdash \varphi$), unless φ becomes true after every admissible plan (for all $\pi' \in \Pi$, $\mathit{state}(\pi') \vdash \varphi$), i.e unavoidable.

Remove Harm

SCIENCES SORBONNE UNIVERSITÉ

Remove Harm

A plan is ethically permissible if it removes existing harm in the initial situation. It requires the agent to mitigate harm whenever possible.

$$\mathcal{EP}_{\mathsf{nnHarm}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi) \text{ and } \mathit{state}(s_0) \vdash \varphi, \text{ then } \big(\mathit{state}(\pi) \nvdash \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\big)\}$$

Remove Harm

Remove Harm

A plan is ethically permissible if it removes existing harm in the initial situation. It requires the agent to mitigate harm whenever possible.

$$\mathcal{EP}_{\text{nnHarm}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi) \text{ and } \mathit{state}(s_0) \vdash \varphi, \text{ then } \big(\mathit{state}(\pi) \nvdash \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\big)\}$$

Explanation:
For every harmful predicate φ, if it is found in the initial state ($\mathit{state}(s_0) \vdash \varphi$), then an acceptable plan π must remove it, or every possible plan must have it at the end (so unavoidable).

Harm Avoidance

SCIENCES SORBONNE UNIVERSITÉ

Harm Avoidance

A plan is ethically permissible if it minimises the amount of harm in the end state.

$$\mathcal{EP}_{\text{harmAvoid}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi), \text{ then } \left(\mathit{state}(\pi) \not\models \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\right)\}$$

Harm Avoidance

SCIENCES SORBONNE UNIVERSITÉ

Harm Avoidance

A plan is ethically permissible if it minimises the amount of harm in the end state.

$$\mathcal{EP}_{\text{harmAvoid}}(\Pi, s_0) = \{\pi \in \Pi \mid \text{for all } \varphi, \text{ if } \mathit{harmful}(\varphi), \text{ then } \left(\mathit{state}(\pi) \not\models \varphi \text{ or for all } \pi' \in \Pi, \ \mathit{state}(\pi') \vdash \varphi\right)\}$$

Explanation:
For every harmful predicate φ, an acceptable plan π must ensure that φ is not true in the end state, or every possible plan must have it at the end (so unavoidable). It is a combination of the two previous principles.

Example

Let's define that the most important thing is to keep the employees safe. So we define the world as:

- $inside(emp, do(a, s)) \equiv inside(emp, s) \wedge a \neq evac$
- $onfire(do(a, s)) \equiv onfire(s) \wedge a \neq extg$
- $injured(emp, do(a, s)) \equiv inside(emp, s) \wedge onfire(s) \wedge a \neq evac$
- $injured' emp, do(a, s)) \equiv injured(emp, s) \wedge a \neq evac \wedge inside(emp, s) \wedge onfire(s)$

Here emp is the employee and $injured(emp)$ the penatly of letting someone get injured. $injured'(emp)$ means even more damage to an employee.

Example

According to our previous definitions, we model the goodness function as:

- $goodness([call]) = -10$
- $goodness([extg]) = -10$
- $goodness([evac]) = 0$
- $goodness([call, evac]) = -10$
- $goodness([evac, call]) = 0$
- $goodness([extg, call]) = -20$
- $goodness([call, extg]) = -20$
- $goodness([evac, extg]) = 0$
- $goodness([extg, evac]) = -10$
- $goodness([call, evac, extg]) = -10$
- $goodness([evac, call, extg]) = 0$
- $goodness([extg, call, evac]) = -20$
- $goodness([call, extg, evac]) = -20$
- $goodness([evac, extg, call]) = 0$
- $goodness([extg, evac, call]) = -10$

Therefore, with the goodness of initial state set to 0, we have:

$$\mathcal{EP}_{util}(\Pi, s_0) = \{[evac], [evac, call], [evac, call, extg], [evac, extg], [evac, extg, call]\}$$

$$\mathcal{EP}_{hedon}(\Pi, s_0) = \{\}$$

Note: Hedonism returns empty set because no plan has a goodness strictly greater than the initial state, which has a goodness of 0.

Formalisation of Properties

Definition

SCIENCES SORBONNE UNIVERSITÉ

Overview

We want to define properties that can be used to **evaluate and compare** ethical principles.
Based on which properties are verified, we can make an **educated choice** on what principle to follow according to the context/situation.

Definition

SCIENCES SORBONNE UNIVERSITÉ

Overview

We want to define properties that can be used to **evaluate and compare** ethical principles.
Based on which properties are verified, we can make an **educated choice** on what principle to follow according to the context/situation.

The authors define 8 properties which are directly derived from the principles we just saw.
7 more are presented, and are derived from properties found in social choice theory (we will define what this means later).

Appendix

Full Example Definition

SCIENCES SORBONNE UNIVERSITÉ

Fluents

- $OnFire(s)$
- $EmployeesSafe(s)$
- $FireExtinguished(s)$
- $FireDepartmentCalled(s)$
- $BuildingDamaged(s)$
- $EmployeesInjured(s)$

Actions

- $extg$
- $evac$
- $call$

Full Example Definition

Initial Definition

- $OnFire(s_0) = True$
- $EmployeesSafe(s_0) = False$
- $FireExtinguished(s_0) = False$
- $FireDepartmentCalled(s_0) = False$
- $BuildingDamaged(s_0) = False$
- $EmployeesInjured(s_0) = False$

Action Precondition Axioms

- $Poss(extg, s) \equiv OnFire(s)$
- $Poss(evac, s) \equiv \neg EmployeesSafe(s)$
- $Poss(call, s) \equiv \neg FireDepartmentCalled(s)$

Positive Effect Axioms

- $\gamma^{+}_{FireExtinguished}(extg, s) \rightarrow FireExtinguished(do(extg, s))$
- $\gamma^{+}_{EmployeesSafe}(evac, s) \rightarrow EmployeesSafe(do(evac, s))$
- $\gamma^{+}_{FireDepartmentCalled}(call, s) \rightarrow FireDepartmentCalled(do(call, s))$

Negative Effect Axioms

- $\gamma^{-}_{OnFire}(extg, s) \rightarrow \neg OnFire(do(extg, s))$
- $\gamma^{-}_{BuildingDamaged}(call, s) \rightarrow \neg BuildingDamaged(do(call, s))$
- $\gamma^{-}_{EmployeesInjured}(evac, s) \rightarrow \neg EmployeesInjured(do(evac, s))$